Progetto 1: Server HTTP Concorrente

Gestione Prenotazione Aule Studio in C

Massimo Mantineo (541924) Pierluca Tino Castorina (545101)

Università degli Studi di Messina
Dipartimento MIFT - Laurea in Informatica

Anno Accademico 2024/2025

Problem Statement: Obiettivi del Progetto

Il Problema

Implementare un server per la gestione di prenotazioni in grado di servire simultaneamente molteplici client, garantendo la coerenza dei dati e riducendo l'overhead computazionale di sistema.

Le Soluzioni Adottate:

- **C Nativo & POSIX:** Utilizzo delle *system call* di basso livello anziché di framework moderni (Node.js, Spring) per massimizzare il controllo delle risorse.
- **Architettura a Thread Pool:** Sostituzione del pattern classico basato su `fork()` con un pool di worker thread pre-allocati.
- **Sincronizzazione Esplicita:** Impiego combinato di Semafori (per la comunicazione di rete) e Mutex (per l'accesso in mutua esclusione al database in memoria).

Stato dell'Arte: Architetture Server a Confronto

Per contestualizzare la scelta progettuale, è utile confrontare i principali paradigmi per la gestione della concorrenza nei server web:

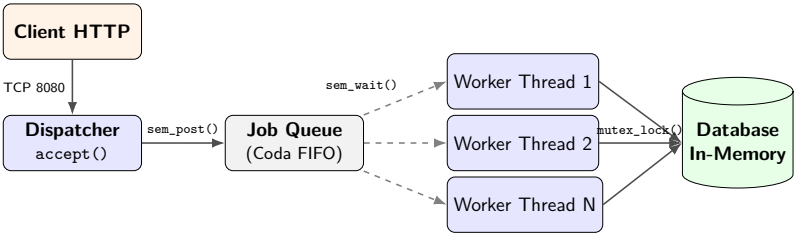
- **Multi-Processo:** Ad ogni richiesta viene associato un nuovo processo tramite `fork()`. Garantisce isolamento totale, ma presenta un elevato overhead di memoria e tempi di *context switch* pesanti.
- **Modello "Thread-per-Connection":** Ad ogni nuova connessione accettata, il server lancia un nuovo thread dedicato tramite `pthreadCreate()`. Sebbene sia più leggero del processo `fork()`, il consumo di memoria cresce linearmente con il numero di client (ogni thread richiede il proprio stack). Inoltre, la creazione e distruzione continua di thread genera un elevato overhead, portando il sistema al collasso in caso di attacchi o carichi improvvisi.
- **Multi-Threading a Pool (Il nostro approccio):** Alloca un numero fisso di Thread (*Worker*) all'avvio. Condivide lo stesso spazio di indirizzamento (memoria condivisa), riducendo drasticamente il consumo di RAM rispetto ai processi, delegando la gestione delle code e la mutua esclusione alle primitive POSIX (Semafori e Mutex).

Metodologia: Lo Stack Tecnologico

Il progetto è stato sviluppato e validato utilizzando esclusivamente strumenti standard e open-source:

- **Build System**: `Makefile` per l'automazione della compilazione.
- **Networking**: `Wireshark` per l'analisi dei pacchetti HTTP/TCP.
- **Benchmarking**: `Apache Bench (ab)` per lo stress test concorrente.
- **Version Control**: `Git` per la gestione del codice sorgente.

Metodologia: Architettura a Thread Pool



Metodologia: Sincronizzazione e Flusso

L'utilizzo dei **Semafori** (`sem_t`) permette di gestire la comunicazione tra thread evitando il *busy-waiting* (polling attivo).

- **Gestione degli Stati**: Il sistema operativo pone i thread worker in *Blocked State* quando la coda è vuota, azzerando il consumo di CPU.
- **Segnalazione (Signal/Wait)**:
  - `sem_wait()`: Decrementa il contatore; se è zero, il thread dorme.
  - `sem_post()`: Incrementa il contatore e risveglia un thread dormiente.

Implementazione: Ottimizzazioni Architetturali

Sincronizzazione Centralizzata (Mutex)

L'architettura previene attivamente *race conditions* e corruzione dei dati in memoria. L'impiego rigoroso del **Mutex** (`pthread_mutex_t`) garantisce che l'accesso alla memoria condivisa avvenga in totale mutua esclusione per ogni Worker Thread, assicurando l'assoluta coerenza transazionale sia in lettura che in scrittura.

De-frammentazione della Memoria Dinamica

La rimozione logica di una prenotazione (`DELETE`) espone al rischio di frammentazione del vettore. L'implementazione impiega uno **shift a sinistra** progressivo tramite un **ciclo iterativo for** all'interno dell'area protetta, garantendo la ricompattazione continua dell'array dinamico senza librerie esterne.

Implementazione: Pattern Produttore-Consumatore

Dispatcher (Produttore)

```
// Accettazione e creazione job
int* client_sock = malloc(sizeof(int));
*client_sock = accept(...);

pthread_mutex_lock(&pool->queue_mutex);
// Aggiunta in coda (FIFO)
pool->queue_tail->next = job;
pool->queue_tail = job;
pthread_mutex_unlock(&pool->queue_mutex);

// Sveglia un thread worker
sem_post(&pool->queue_sem);
```

Worker (Consumatore)

```
while (1) {
    // Attesa passiva (nessun busy-waiting)
    sem_wait(&pool->queue_sem);

    pthread_mutex_lock(&pool->queue_mutex);
    if (pool->shutdown) {
        pthread_mutex_unlock(&pool->
            queue_mutex);
        break; // Shutdown controllato
    }
    // Estrazione job dalla coda...
    pthread_mutex_unlock(&pool->queue_mutex);

    handle_client(job);
}
```

Implementazione: Thread Safety sui Dati

L'accesso alla risorsa condivisa (il vettore delle prenotazioni) è protetto da un Mutex per garantire la coerenza dei dati durante operazioni concorrenti.

```
int rm_create(resource_manager_t* rm, const char* room, ...) {
    pthread_mutex_lock(&rm->mutex); // ACQUISIZIONE LOCK

    if (rm->count >= MAX_RESERVATIONS) {
        pthread_mutex_unlock(&rm->mutex);
        return -1;
    }

    // Scrittura sicura nell'array condiviso
    strncpy(rm->reservations[rm->count].room, room, 127);
    rm->count++;

    pthread_mutex_unlock(&rm->mutex); // RILASCIO LOCK
    return rm->count - 1;
}
```

Implementazione: Eliminazione Atomica

Le operazioni logiche complesse, come la rimozione con ricompattazione (shift), devono avvenire integralmente sotto lock.

```
int rm_delete(resource_manager_t* rm, int id) {
    pthread_mutex_lock(&rm->mutex); // Sezione Critica
    for (int i = 0; i < rm->count; i++) {
        if (rm->reservations[i].id == id) {
            // Shift a sinistra per ricompattare l'array
            for (int j = i; j < rm->count - 1; j++) {
                rm->reservations[j] = rm->reservations[j + 1];
            }
            rm->count--;
            pthread_mutex_unlock(&rm->mutex);
            return 0;
        }
    }
    pthread_mutex_unlock(&rm->mutex);
    return -1;
}
```

Implementazione: Shutdown Sicuro

La chiusura del server richiede un coordinamento per evitare *deadlock* o thread rimasti in attesa sul semaforo.

```
void thread_pool_destroy(thread_pool_t* pool) {
    pthread_mutex_lock(&pool->queue_mutex);
    pool->shutdown = 1; // Segnale di stop
    pthread_mutex_unlock(&pool->queue_mutex);

    // Svegliamo tutti i thread in attesa su sem_wait()
    for (int i = 0; i < pool->num_threads; i++) {
        sem_post(&pool->queue_sem);
    }

    // Join per attendere la terminazione effettiva
    for (int i = 0; i < pool->num_threads; i++) {
        pthread_join(pool->threads[i], NULL);
    }
}
```

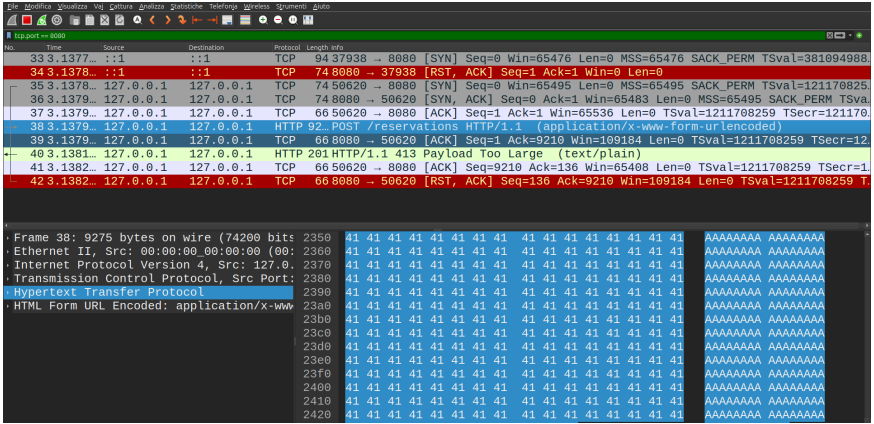
Implementazione: Specifiche API REST

Metodo	Endpoint	Operazione nel Backend C	Success Code
`GET`	`/reservations`	Mutex Lock → Serializzazione JSON → Unlock	`200 OK`
`POST`	`/reservations`	Parsing URL-Encoded, Aggiunta struct in array	`201 Created`
`PUT`	`/reservations/{id}`	Ricerca lineare ID e aggiornamento campi	`200 OK`
`DELETE`	`/reservations/{id}`	Memory shift a sinistra e ricompattazione array	`204 No Content`

Implementazione: Sicurezza e Controlli

Mitigazione delle criticità intrinseche in ambiente C/POSIX:

- **Socket Leaks:** Dismissione sicura `close(fd)`.
- **Buffer Overflows:** Varianti sicure, errore 413.
- **Interruzione SIGPIPE:** Flag `MSG_NOSIGNAL`.



Implementazione: Networking e Handshake TCP

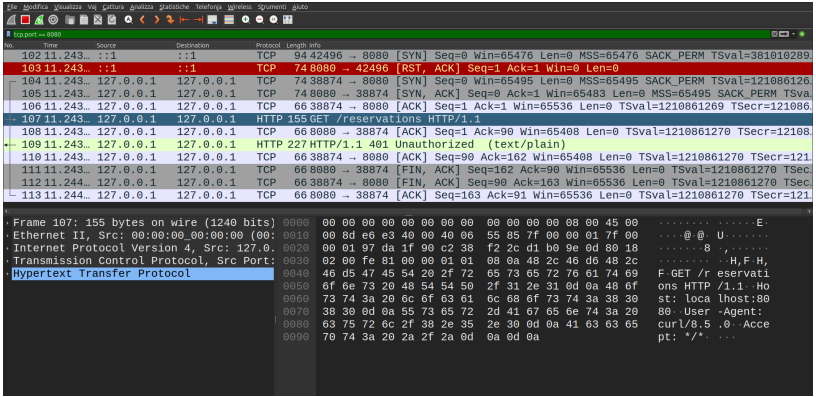
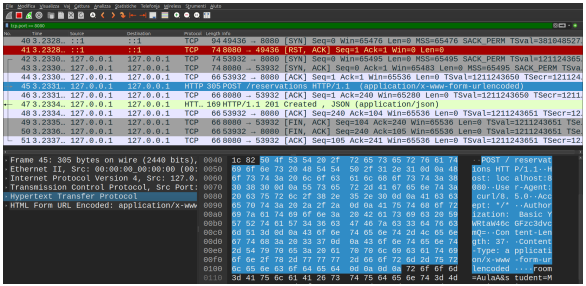


Figura: Tracciato Wireshark del Three-Way Handshake TCP.

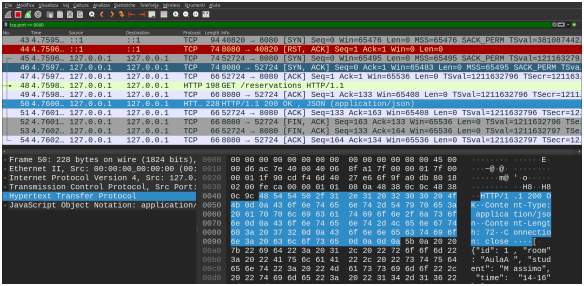
Autenticazione

Le rotte sensibili richiedono l'header `Authorization: Basic` (token Base64).

Implementazione: Analisi Traffico HTTP



POST Request e Basic Auth



Risposta 200 OK (JSON)

Risultati: Load Testing e Performance

Il progetto è stato collaudato sfruttando lo strumento standard **Apache Bench (ab)**. La simulazione ha previsto: **10.000 richieste totali** con un livello di concorrenza di **100 client paralleli**.

Metriche di Prestazione

- **Total Requests:** 10.000
- **Concorrenza Esterna:** 100
- **Success Rate:** 100% (0 Errori)
- **Throughput:** $\sim$ 33.229 Req/Sec

Interpretazione Architetturale

L'assenza totale di connessioni droppate dimostra che la coda semaforica regge il carico e previene i fallimenti TCP. Inoltre, il tempo di risposta mediano di appena **3.0 ms** per richiesta conferma la leggerezza dei thread worker e la corretta implementazione dei lock Mutex sulle risorse condivise.

Conclusione e Sviluppi Futuri

L'implementazione in linguaggio C di un Thread Pool associato a socket BSD si è dimostrata una soluzione *memory-safe* ad altissime prestazioni per la costruzione di architetture HTTP concorrenti.

Sviluppi per Ambienti di Produzione:

1. **Persistenza dei Dati**: Passaggio dalla struttura in RAM ad un database relazionale leggero (come `SQLite3`) per mantenere lo stato dopo il riavvio del processo.
2. **I/O Asincrono (`epoll`)**: Sostituzione delle chiamate bloccanti `read()` con un loop *Event-Driven* nativo Linux per supportare il limite dei 10.000 Socket attivi.
3. **Cifratura**: Implementazione del protocollo *TLS/SSL* tramite librerie come *OpenSSL* per esporre API sicure (HTTPS).

Grazie per l'attenzione.